

CLASS 3 · PART A

# Obsidian, Claude Code & the *LLM Wiki*

*Knowledge management that compounds.*

Jiaqi Shao · NUS Business School

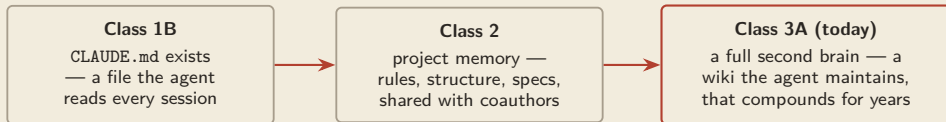


## ■ Where We Are in the Course

---

*Today, course message three reaches its endgame.*

Three messages run through this course: *agentic partner* · *context engineering* · **persistent memory**  
— today completes the third.



---

**Carried from Class 1A: hallucination · automation bias ·**  
**“Could another researcher reconstruct the evidence path?”**

## ■ Today: A Second Brain That Compounds

---

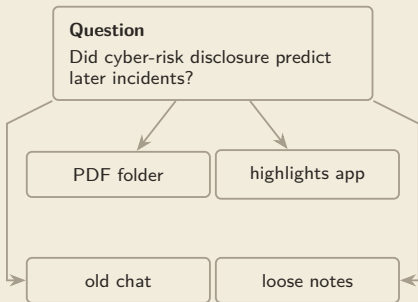
*Not another note app — a system where knowledge accumulates instead of rotting.*



- **Problem** — why notes rot, and the physical reason an AI alone can't fix it.
- **Pattern** — one design idea (Karpathy) + the open-source project that implements it to the letter.
- **Tools** — Obsidian, Claude Code, CLAUDE.md — each with exactly one job.
- **Rhythm** → **Hands-on** — daily one command, weekly one question — then 35 protected minutes building a mini vault on *your own* papers.

## ■ Six Months Later, Can You Recover the Evidence?

*You remember the conclusion from Smith (2024) — but not why you believed it.*



### Four failures appear together

- **Evidence is unrecoverable** — which page supports the claim?
- **Materials are scattered** — which copy is authoritative?
- **Synthesis evaporates** — what did you conclude across papers?
- **Conflicts disappear** — did Jones (2025) overturn it?

**The need is not “better notes.” It is a memory that preserves evidence, synthesis, links, and disagreement.**

## ■ A Research Memory Has Four Requirements

---

*A useful system must answer the Smith (2024) question without trusting recollection.*

### Evidence

claim →  
source → page

### Synthesis

what we learned  
across sources

### Links

papers, concepts,  
entities

### Disagreement

conflicts remain visible  
until a human rules

A model session is temporary: its working context ends, and useful synthesis is not automatically written into your research record.

So persistent memory must live in inspectable external artifacts. `CLAUDE.md` was our first one; today we build the full system.

---

**context window** — the temporary working memory available in one model interaction

## ■ RAG Solves Retrieval: Bring Relevant Evidence Back

*For the Smith (2024) question, retrieve the passages most likely to contain the answer.*

**RAG** — Retrieval-Augmented Generation: retrieve relevant material from an external collection and place it in the model's context before it answers. A persistent search index is one common implementation.

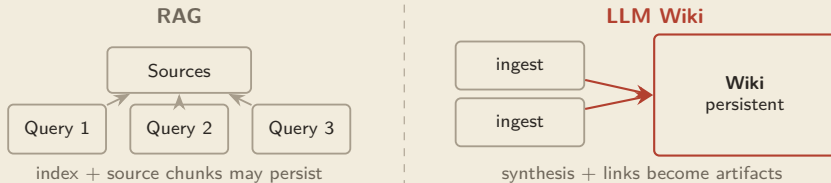


This is genuinely useful for lookup. The next problem is different: who records the synthesis, links, and decisions so the next project can reuse them?

**retrieval** — selecting the external passages most relevant to the current question

## ■ Retrieval and Maintained Synthesis Are Different Jobs

*Retrieval brings evidence back; wiki maintenance preserves what the research learned.*



- **Left:** RAG keeps sources or an index available and retrieves relevant passages; the answer's synthesis is not automatically added to a maintained knowledge model.
- **Right:** the wiki deliberately writes reusable synthesis, links, provenance, and decisions for later work.

**RAG retrieves evidence; the wiki maintains what the research process learned.**

## ■ Part I — Conclusion

---

*Why a maintained wiki is necessary, not just nice.*



The open question: **who maintains it?** Not you — that's the rot. Next: *the agent* as maintainer.

**Make the research record remember: persist synthesis, links, provenance, and decisions in inspectable artifacts.**





## ■ Concept Check — Name the Two Jobs

---

*Before we meet the pattern: say the why out loud.*

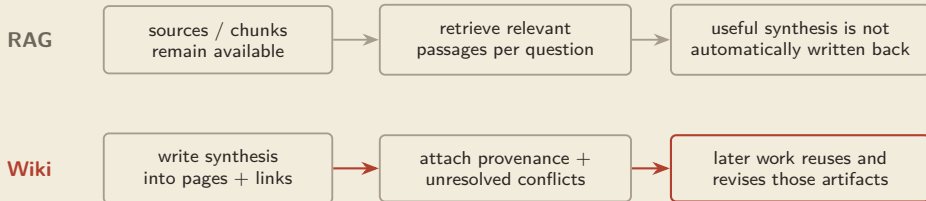
**In one sentence: what does retrieval preserve,  
and what does a maintained wiki add?**

Write your sentence down before the next slide.

Hint: distinguish source/index artifacts from synthesis, links, provenance, and decisions.

## ■ Concept Check · Answer

*The systems can coexist because they preserve different artifacts.*



**Retrieval brings evidence into context; a maintained wiki deliberately writes reusable synthesis, links, provenance, and decisions back to disk.**

PART II

# The LLM-Wiki Pattern

- *one design idea, three layers, three operations.*



## ■ The LLM-Wiki Pattern

*Separate raw sources from synthesised knowledge — and let the LLM maintain the second.*

- Origin. Andrej Karpathy, “LLM Wiki” (public gist, April 2026) — a pattern for personal knowledge bases; viral within days.
- The gist is written to be *handed to your own LLM agent* — the pattern assumes an agent as maintainer from the first line.

### Division of labour

**The human** — selects sources, judges conflicts, approves consequential claims

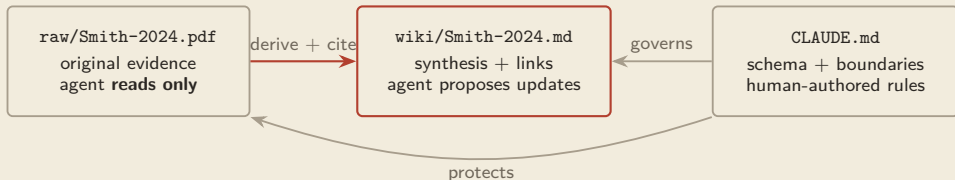
**The LLM** — proposes summaries, links, and updates at scale

**Schema + lint** — constrain format and surface maintenance problems for review

**Result: a knowledge base that is *persistent and compounding*.**

## ■ Three Layers Separate Evidence, Interpretation, and Rules

*Who owns what — the whole design in one picture.*



- **raw/** — immutable papers, articles, images, data; never edited.
- **wiki/** — summaries, entities, concepts, overview; agent-maintained under review.
- **CLAUDE.md** — the *schema* (plain English for rulebook): structure, naming, ingest & Q&A workflow.

**raw/ is read-only · wiki/ is agent-maintained · the schema governs both.**

## ■ Why raw/ Is Never Edited

---

*The raw folder is your evidence chain.*

wiki pages are the agent's *interpretations* → interpretations can be wrong or go stale  
→ so you must always be able to re-derive them from untouched originals

### Class 1A's audit question

"Could another researcher reconstruct the evidence path?"

An immutable **raw/** makes reconstruction possible; provenance fields make the path usable.

---

**If raw/ is intact, no agent mistake is permanent.**

## ■ Operation 1 — Ingest Smith (2024)

---

*One new source in; the whole wiki updated around it.*



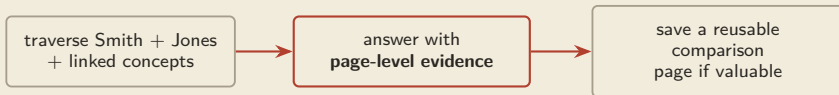
The paper is not merely filed. Its claims become traceable, linked research artifacts — while Smith's PDF remains untouched.

---

**Ingestion is not filing — it is integration: new knowledge is linked into old.**

## ■ Operation 2 — Query Across Smith (2024) and Jones (2025)

*Answers come with citations — or they don't count.*



The answer must distinguish “Smith reports” from “we infer.”

A valuable comparison may be saved, but only with its evidence path and unresolved conflicts.

### **Class 1A, Output Risk → remedy: source verification**

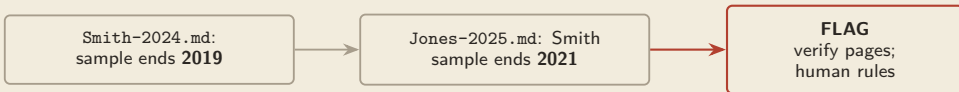
Here the remedy is built into the system: every claim points to a page; every page points to **raw/**.



## ■ Operation 3 — Lint the Smith–Jones Conflict

*A scheduled health check for your knowledge.*

**lint** (from programming) = an automated check that finds problems before they hurt you.



**Stale claims are Class 1A's Information-Set Risk** — in your own notes: what was true when written may not be true now. Lint fights it on a schedule, not by luck.

**The agent can scan everything monthly; humans review the consequential flags.**



## ■ Two Support Files Do the Bookkeeping

*No embeddings, no database — a contents page and a timeline.*

### index.md

table of contents — one line per page, grouped by category;  
find anything without a search index

```
## Concepts
- [[disclosure-theory]]
## Entities
- [[SEC]] - regulator
```

### log.md

append-only timeline — one parseable line per operation

```
## [2026-07-30] ingest | Smith 2024
## [2026-07-30] lint | 2 stale
```

Anything can be found by *reading*, not by similarity search — if you can write a shopping list, you can read both files.

**wikilinks** — `[[double brackets]]` around a note's name turn it into a link

## ■ log.md Is Your Audit Trail

*The written answer to “what exactly did the system do?”*

---

### Audit question (Class 1A)

Where the answer lives

What exactly did the system do?

log.md, line by line

Where is the record of its action?

log.md + dated lint reports

Could another researcher reconstruct the evidence path?

log.md + immutable raw/

---

Least privilege must be configured and checked: instructions alone do not physically prevent the agent from editing raw/.

---

**Delegation Risk's remedy is audit trails — here, the trail writes itself.**

## ■ Retrieval and Wiki Maintenance Can Work Together

---

*Same sources, complementary jobs: retrieve evidence and maintain synthesis.*

|                   | LLM Wiki                      | RAG                                  |
|-------------------|-------------------------------|--------------------------------------|
| Primary job       | maintain research knowledge   | retrieve relevant evidence           |
| Persistent object | synthesis + links + decisions | sources and often a search index     |
| Quality control   | schema + lint + human review  | retrieval evaluation + source checks |
| Finding things    | index + links + traversal     | keyword / vector / hybrid search     |
| Best fit          | cumulative research programs  | lookup across a document collection  |

---

**Use retrieval to find evidence; use the wiki to preserve and revise what the research learned.**



## ■ Quiz 1 — Approve or Deny?

---

*A real approval prompt, mid-lint.*

Lint found: `[[Smith-2024]]` summary says “sample ends 2019” but the clipped article states 2021.

**Proposed action:** `edit raw/Smith-2024-clipping.md` to correct the year.

**Allow? / Deny?**

Vote: approve or deny — and name what tipped you off.



## ■ Quiz 1 · Answer: Deny

---

*The path is the tell — raw/ is never edited.*

- **Deny.** The intent is fine; the *path* is not — raw/ is immutable, no exceptions.
- The fix belongs in the wiki: update [[Smith-2024]] and record the discrepancy — “clipping says 2021; earlier summary said 2019.”
- If a source itself is wrong, the wiki *notes that it is wrong* — it never rewrites the evidence.

**Same skill as Class 1B's approval-gate quiz:** read paths, not just intent.

An edited source can never again settle a dispute.



## ■ Textbook Case: ekadetov/llm-wiki

---

*None of this is hypothetical — the whole pattern, on disk, in five entries.*

```
<wiki>/  
├── raw/  (articles + attachments)  
├── wiki/ (index.md, concept & entity  
│       │ pages, saved queries)  
├── outputs/reports/ (dated lint reports)  
├── CLAUDE.md  
└── log.md
```

### Command strip

```
wiki init · ingest · compile · query  
· lint
```

every command maps to a concept from  
Part II

---

**This tree is your mental template for the hands-on.**

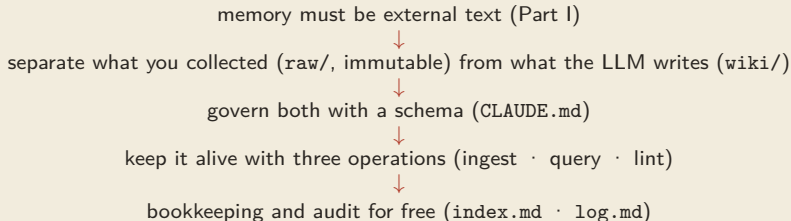
one of a wider ecosystem of implementations → backup slides

Small, readable codebase — a pattern exemplar, not a product. Verified live 2026-07-30.

## ■ Part II — Conclusion

---

*The pattern in one chain.*



**Everything you do to the wiki is ingest, query, or lint —  
and none of them ever edits raw/.**



PART III

# Division of Labour

- *Obsidian for you · Claude Code for the work · CLAUDE.md for the rules.*



## ■ Obsidian — the Human Interface

---

*Where you read, browse, and edit — in a format you permanently own.*

- Graph view + backlinks

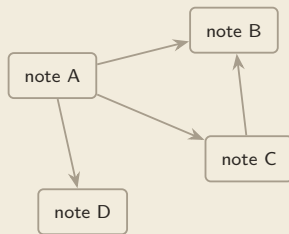
see how notes connect.

- `[[wikilinks]]`

double brackets around a name make a link.

- Plain Markdown files

no lock-in; readable in any editor even if Obsidian disappears.



graph view

Free for personal use.

## ■ Claude Code — the Agent

*It reads and writes your Markdown natively — no plugin, no API glue.*



Four spokes = the three operations, plus the repairs lint recommends.

The vault is just `.md` files on disk — exactly what the agent already reads and writes.

**Nothing new to install beyond Class 1B's setup.**

## ■ CLAUDE.md — the Schema

---

*Write down once what you'd otherwise re-explain every session.*

```
# Research Wiki - schema
raw/  read-only sources; never edit
wiki/ you maintain: summaries, entities, concepts
Frontmatter: tags, type, status, created, updated
Make a [[wikilink]] when a concept recurs
After every ingest: update index.md + log.md
```

- Auto-loaded at session start as *context* — read like a brief, not rigid config.
- Same file you built in Class 2 — now governing knowledge instead of code.

---

**frontmatter** — the labelled block at the top of a note holding tags, type, status, dates



## ■ What Earns a Line in the Schema

---

*Only what every session needs — the Class 2 rule still applies.* The test, from Class 2: **does every session need this line?** Five yeses:

- folder map and what's read-only
- frontmatter fields (tags / type / status / dates)
- note format for each page type
- when to create a `[[wikilink]]`
- when to update `index.md` and `log.md`

**NOT:** task-specific procedures (→ a skill) · step-by-step how-to (→ the agent's job) · anything you wouldn't want re-read every single session

## ■ Provenance Turns a Claim into an Auditable Path

*Without source and location, "Smith finds 25%" is a memory — not evidence.*

**At the top of** Smith-2024.md

```
---  
type: literature-note  
status: needs-review  
source: raw/Smith-2024.pdf  
pages: 18--19  
updated: 2026-07-30  
---
```

### Claim

Disclosure predicts later incidents  
by 25%.



### Metadata

source + pages + review status



### Re-check

open pp. 18–19; verify outcome  
and denominator

**Frontmatter locates the evidence; it does not validate the interpretation.**

## ■ Building Block 1 — Claude Code Memory

---

*Two documented mechanisms carry knowledge across sessions.*

| CLAUDE.md              | Auto memory                     |
|------------------------|---------------------------------|
| <i>you</i> write it    | <i>the agent</i> writes it      |
| instructions and rules | learnings from your corrections |
| loaded every session   | loaded every session            |

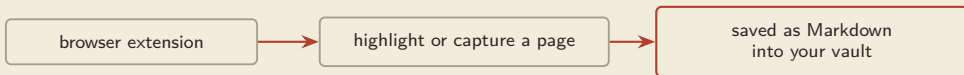
Both are the official answer to “each session begins with a fresh context window” — the docs’ own framing of Slide 5’s physical fact.

[code.claude.com/docs/en/memory](https://code.claude.com/docs/en/memory)

## ■ Building Block 2 — Obsidian Web Clipper

---

*Capture is one click; the clipping lands in raw/.*



The moment saving a source costs effort, the inbox habit dies — and the whole pipeline starves.  
One click feeds raw/; the agent does everything after.

---

**Capture must be frictionless — or the habit dies.**

`obsidian.md/clipper`



## ■ Why an LLM? Semantic Understanding

*Keyword tools match words; the agent reads meaning.*

| Filing sentence                                     | Keyword count                  | The agent                   |
|-----------------------------------------------------|--------------------------------|-----------------------------|
| A: "We face material cybersecurity risk"            | flags "cyber risk"             | live risk disclosure        |
| B: "We have fully addressed our cybersecurity risk" | flags "cyber risk" (identical) | mitigation claim (opposite) |

**keyword NLP** — classic text tools that count and match words without reading meaning.

**Semantic reading makes summaries, link suggestions, and conflict flags possible;  
evidence checks and human review make them trustworthy.**

## ■ When NOT to Use the Agent

---

*If a keyword search solves it, don't send a language model.*

### Cheap tools win

- count word frequencies
- find an exact phrase across files
- rename / batch-move files

### The agent wins

- summarise what a paper argues
- decide which notes a new source relates to
- detect that two pages disagree

**If a regex or a word-count solves it, don't send a language model.**

(Same rule as Class 1B — it applies to your notes too.)

## ■ Quiz 2 — Which Lines Belong in CLAUDE.md?

---

*Three candidate lines — vote on each.*

- (a) Frontmatter: every note carries tags, type, status, created
- (b) To parse PDFs: first run pdftotext, then split on section headers, then for each section... (12 more steps)
- (c) Never edit raw/; summaries and corrections go in wiki/

For each: schema — or somewhere else? Vote a / b / c in turn.



## ■ Quiz 2 · Answer

---

*The schema holds boundaries and formats — not procedures.*

- (a) **YES** — a format every session needs.
- (c) **YES** — a boundary every session needs.
- (b) **NO** — a task procedure: it belongs in a *skill* (loaded only when parsing), and its step-by-step detail micromanages the agent.

CLAUDE.md carries only what every session needs: boundaries, formats, workflow one-liners.  
Procedures go in skills; steps belong to the agent.

PART IV

# The Working Rhythm

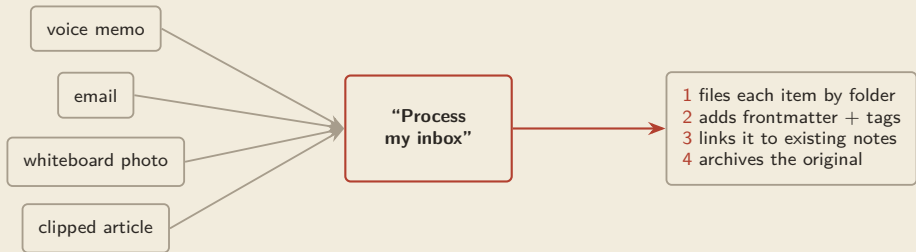
■ *daily: one command · weekly: one question · then the tradeoff.*



## ■ Daily Workflow: “Process My Inbox”

*Capture without sorting; one command files everything.*

All day you dump into one Inbox — you do *not* sort: discipline doesn't scale, **scheduling does**.



**You capture all day, then say three words — filing happens to *your* rules.**

## ■ Weekly: Ask for the Non-Obvious

---

*Once knowledge is connected, ask what no single note can answer.*

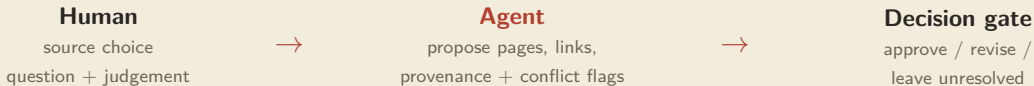
*“What non-obvious **connections**, **patterns**, or **contradictions** exist across my notes this week?”*

- Synthesis, not retrieval — the agent reads across many pages at once.
- Surfaces links you didn't notice and claims that disagree.
- Turns a pile of notes into a line of argument — most of what a literature review is.

## ■ The Human–Agent Handoff Has a Decision Gate

---

*The agent proposes maintenance; consequential interpretation returns to you.*



🔄 The human ruling is written back to the wiki / log.

- The agent proposes maintainable **artifacts**: pages, links, provenance fields, and dated reports.
- The human owns contested interpretation: approve, revise, or explicitly keep the Smith–Jones conflict unresolved. The ruling returns to a file for later audit.

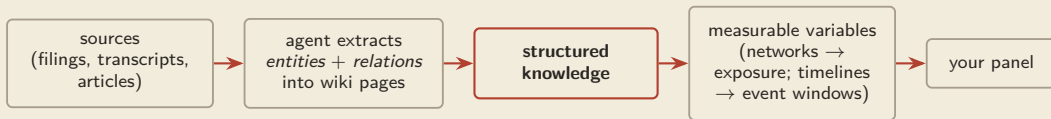
---

**The agent proposes; the human rules; the file preserves both the evidence and the decision.**



## ■ The Endgame: From Wiki to Research Variables

*Structured knowledge can feed empirical work directly.*



**entity** — a thing your research tracks (a firm, a person, a regulator, an event); structured knowledge *computes*.

A direction, not an assignment — the same three operations, aimed at data instead of notes.

Same skeleton as the course's text-analysis workflow —  
the wiki is the organised, auditable middle layer.



## ■ The Build Path: Prototype → Codify → Scale

---

*Debug interactively on five documents before you trust a hundred.*

- **1 Prototype** — work interactively in Claude Code on 5–10 documents; catch the misfiled, the mis-parsed, the mis-linked.
- **2 Codify** — write what you fixed into the schema (or a skill): the correction becomes a rule.
- **3 Scale** — only then run the whole backlog or a scheduled agent.

Every correction you type twice belongs in the schema.

## ■ Tradeoffs: Plugins vs. the Agent

*Turnkey and private, or powerful and demanding — pick for the job.*

|             | Plugins (chat / related-notes —<br>named in backup) | Claude Code (the agent)           |
|-------------|-----------------------------------------------------|-----------------------------------|
| Setup       | turnkey                                             | needs CLAUDE.md + discipline      |
| Privacy     | can run fully local                                 | sends what it reads to the model  |
| Capability  | chat / find-similar                                 | read / write / restructure        |
| Maintenance | you do it                                           | agent maintains structure & links |

**Plugins find; the agent maintains.**

Confidential material → local plugin or check policy; the compounding wiki → the agent.

## ■ Quiz 3 — The Colleague's Question

---

*You will be asked this. Which reply survives scrutiny?*

A colleague sees your vault: *"Isn't this just RAG with extra steps?"*

- A** — "No — RAG is obsolete; the wiki replaces it."
- B** — "Basically yes — both let the AI answer from my files."
- C** — "Different jobs. RAG retrieves relevant evidence from sources or an index. The wiki deliberately maintains synthesised pages, links, provenance, and decisions — so research knowledge accumulates. They can be combined; for one-off lookup, retrieval may be enough."

Vote A / B / C.



## ■ Quiz 3 · Answer: Only C Survives

---

*State the mechanism; concede the boundary.*

- **A overclaims** — RAG isn't obsolete; for one-off lookup over documents you'll never revisit, it's the right tool. One follow-up question sinks you.
- **B underclaims** — it misses the whole mechanism: persistence, citations, lint. You'd forfeit the compounding for a false equivalence.
- **C survives** — names the mechanism (maintained synthesis vs. evidence retrieval), the boundary, and the fact that they can be combined.

---

Same discipline as Class 1B's compliance quiz:  
the defensible answer states both halves precisely.

## ■ Part IV — Conclusion

---

*The rhythm fits on a sticky note.*

**Daily** — capture all day, then: “process my inbox”

**Weekly** — one question: connections, patterns, contradictions

**Monthly / scheduled** — lint, and read the report

**Capture is one click; filing is one command;  
synthesis is one question.**

PART V

# Build Your Mini Research Vault

■ 35 minutes · 2–3 of your own papers · everything from today.





## ■ Setup Check — 2 Minutes

---

*Three green lights, then we build.*

- **1** a terminal where `claude` starts  
(Class 1B setup — nothing new needed)
- **2** a fresh empty folder, e.g. `my-wiki/`, with two subfolders: `raw/` and `wiki/`
- **3** 2–3 PDFs you know well, copied into `raw/`

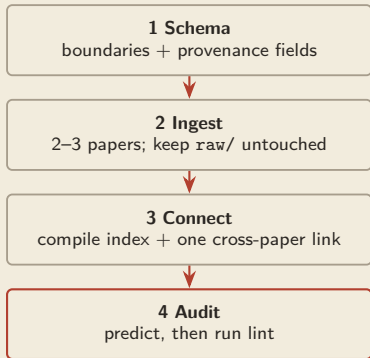
No paper handy? Any PDF works — grab one from a preprint server.

Obsidian is optional today: the vault is just files.



## ■ Build One Complete Evidence Loop

*Your success criterion: recover one claim from a wiki page back to the PDF location.*



### Minimal schema starter

```
raw/ is read-only; never edit
wiki/ holds literature + concept notes
Frontmatter: source, pages, status, updated
Link recurring concepts
Update index.md + log.md after ingest
```

Try: Read CLAUDE.md, ingest @raw/Smith-2024.pdf, update index and log, then show the evidence path for each claim.

## ■ Worked Example: One Ingest, Narrated

---

*Watch the four artifacts appear before you build.*

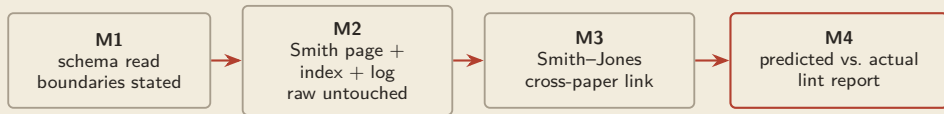
```
wiki/Smith-2024.md  new summary page - frontmatter filled,
                    [[wikilinks]] to recurring concepts
index.md            one new line under Papers
log.md              one new line:
                    ## [2026-07-30] ingest | Smith 2024
raw/Smith-2024.pdf  untouched
```

---

**During the run, read every approval prompt aloud: path first, then intent.**

## ■ Four Milestones Make Progress Observable

*Four checkpoints; write your expectation before the lint.*



**Before M4, write what lint should find.**

Example: "Smith and Jones should link via [[cyber-risk]], and their sample-end conflict should remain flagged."

**A milestone is an observable artifact — not "the agent says it is done."**

## ■ What Good Looks Like

---

*A clean index and an honest lint report — the two signs it's working.*

index.md

```
# Index
## Concepts
- [[disclosure-theory]]
## Entities
- [[SEC]] - 12 links
## Papers
- [[Smith-2024]] - cyber-risk
```

lint report

```
Lint - 2026-07-30
OK    42 pages, 128 links
WARN  2 stale claims flagged
WARN  1 orphan: [[old-draft]]
FIX    add ref: Smith-2024 -> SEC
```

---

**An honest report flags problems — warnings mean lint is *working*, not failing.**

## ■ If It Breaks — Nothing Is Lost

---

*raw/ is intact, so every mistake is recoverable.*

- error you don't understand → paste it back: "this failed, fix it"
- agent misfiled or mangled a page → "delete that page and re-ingest from raw/"
- wiki is a mess → delete wiki/ entirely, keep raw/ + CLAUDE.md, start clean in 5 minutes

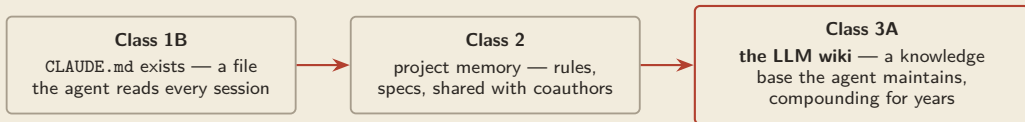
**This is why raw/ is immutable: the vault is rebuildable;  
only the sources are sacred.**

(Quiz 1, vindicated.)

## ■ The Endgame of Persistent Memory

---

*Course message three, complete.*



---

**One mechanism the whole way: external text, re-injected every session.**  
**The wiki is CLAUDE.md's destiny, not a new idea.**



## ■ Preview: Evidence Discipline, Packaged

---

*The practice kit ships the same discipline as your wiki — for papers.*

- **faithfulness.md** — a rule file: every number and claim traces to a section or table; “not reported” beats a plausible guess; *shows  $\neq$  claims  $\neq$  interprets*.
- **the referee agent** — a reviewer in a fresh context: credit first, every concern cites what it attacks, verdicts with load-bearing reasons.

From the course practice kit: `Practice_Jiaqi/claude-setup`.

Your wiki already enforces cite-or-it-doesn't-count.

Next: aim that discipline at a real paper — talk, referee report, and a wiki, from one manuscript.

## ■ Three Things to Remember

---

*If you keep three ideas from today:*

1

### **Separate raw from wiki.**

Sources are evidence — immutable;  
knowledge is interpretation —  
maintained.

2

### **The schema is the work.**

One page of rules turns upkeep from a  
chore into an automatic habit.

3

### **Only maintained memory compounds.**

Retrieval preserves access to evidence;  
the wiki preserves and revises synthesis.

- *Build the vault; let it grow with you.*





## ■ After Today

---

*One week, one habit, then the next class.*

- **Study notes** — this session's companion notes (concepts, steps, quiz answers, links) are in `Slides_Jiaqi/notes/` — don't transcribe, they're written down.
- **This week** — keep the vault from the lab alive: clip 3 sources, run “process my inbox” daily, ask the weekly question on Friday.
- **Next: Class 3 · Part B** — LLM use cases in empirical research (Shirley) — your wiki is where those readings should land.

## ■ References

---

-  A. Karpathy. *LLM Wiki*. Public gist, April 2026.  
`gist.github.com/karpathy/442a6bf555914893e9891c11519de94f`.
-  Anthropic. *How Claude remembers your project*. `code.claude.com/docs/en/memory`.
-  Obsidian. *Web Clipper*. `obsidian.md/clipper`.
-  Open-source implementations: AgriciDaniel/claude-obsidian · logancyang/obsidian-copilot · brianpetro/obsidian-smart-connections · eugeniughelbur/obsidian-second-brain · ballred/obsidian-claude-pkm · ekadetov/llm-wiki.

All URLs and repositories verified live 2026-07-30; star counts approximate and moving.

CLASS 3 · PART A

# Thank you.

*Start small: one schema, three papers.  
Let the wiki compound.*



## ■ Backup: ekadetov/llm-wiki Commands

---

| Command                                 | What it does                                   |
|-----------------------------------------|------------------------------------------------|
| <code>wiki init</code>                  | scaffold the vault + the CLAUDE.md schema      |
| <code>wiki ingest &lt;source&gt;</code> | read, summarise, update pages + index.md       |
| <code>wiki compile</code>               | rebuild index.md from current pages            |
| <code>wiki query "&lt;q&gt;"</code>     | answer with citations to wiki pages            |
| <code>wiki lint</code>                  | health check; dated report to outputs/reports/ |

Interfaces evolve — verify against the current plugin.

## ■ Backup: A Frontmatter Schema

---

```
---  
tags: [disclosure, cyber-risk]  
type: literature-note  
status: read  
created: 2026-07-30  
updated: 2026-07-30  
source: [[raw/Smith-2024.pdf]]  
---
```

Define once in **CLAUDE.md**; the agent fills it on every ingest.  
Consistent frontmatter is what makes Obsidian's Dataview and search work.

## ■ Backup: The Wider Ecosystem — Six Verified Projects

*All checked on GitHub — each teaches one piece of the pattern.*

| Repository                            | Stars  | What it teaches                                                               |
|---------------------------------------|--------|-------------------------------------------------------------------------------|
| AgriciDaniel/claude-obsidian          | ~10.1k | self-organising vault; PARA / Zettelkasten / LYT; built on Karpathy's pattern |
| logancyang/obsidian-copilot           | ~7.5k  | in-Obsidian AI chat + vault Q&A (the plugin route)                            |
| brianpetro/obsidian-smart-connections | ~5.3k  | local-embedding related-notes; no API key (the privacy route)                 |
| eugeniughelbur/obsidian-second-brain  | ~3.7k  | persistent memory for CLI agents; scheduled overnight maintenance             |
| ballred/obsidian-claude-pkm           | ~1.7k  | complete beginner starter kit                                                 |
| ekadetov/llm-wiki                     | ~0.1k  | the textbook Karpathy implementation                                          |

Star counts approximate, as of 2026-07-30. PARA / Zettelkasten / LYT = popular note-organisation methods.

## ■ Backup — Production Case: claude-obsidian (~10.1k★)

---

*The same pattern, grown into a full kit.*

- Drop any source — the agent reads, links, and files it into one connected graph of plain Markdown.
- Supports the classic organisation methods: PARA, Zettelkasten, LYT.
- Its own description: “based on Karpathy’s LLM Wiki pattern.”

**What it teaches:** the pattern scales from a five-file demo to a daily-driver system *without changing shape*.

Stars approximate, as of 2026-07-30; verified live on GitHub.



## ■ Backup — Automation Case: obsidian-second-brain (~3.7k★)

---

*Maintenance you can schedule is maintenance that happens.*

- Persistent memory for Claude Code and other CLI agents — stop re-explaining your projects every session.
- Hybrid semantic search + self-updating notes.
- **Scheduled agents maintain the vault overnight** — lint while you sleep.

**What it teaches:** the rot-killer isn't discipline, it's *scheduling* — upkeep that no longer depends on you remembering.

Stars approximate, as of 2026-07-30; verified live on GitHub.





## ■ Backup — Pick by Need: Two Routes, One Pattern

---

*Repos rise and fall; the skeleton transfers.*

### Plugin route (turnkey, can stay local)

- Copilot — chat
- Smart Connections — related notes

### Agent route (the full pattern)

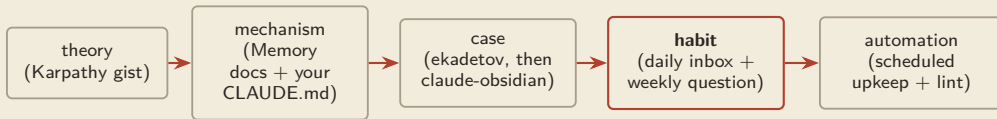
- ekadetov — learn the shape
- ballred — starter kit
- claude-obsidian — daily driver
- second-brain — scheduled automation

**Study the smallest, adopt the pattern — not the repo.**

The skeleton (raw / wiki / schema / three operations) is the transferable asset.

## ■ Backup: Where to Go After Today

---



---

**Habit comes before automation.**



## ■ Backup: Privacy & Policy

---

- What the agent reads is sent to the model API — no bulk upload, no stored cloud index (Class 1B's honest answer).
- Fully-local alternative for confidential material: Smart Connections (local embeddings, no API key).
- Restricted data → institutional policy first; enterprise options exist.